Slovenská Technická Univerzita v Bratislave

Fakulta informatiky a informačných technológií

FIITMeteo – Návrh a implementácia komunikačného protokolu

Počítačové a komunikačné siete

Ondrej Nebus

AIS ID: 133751

Vedúci cvičenia: Roman Gajdoš
Čas cvičenia: Štvrtok 10:00 – 11:50

Obsah

Úvod a architektúra riešenia **3**

Návrh komunikačného protokolu **3**

Identifikácia a sekvenčné čísla 4

Typy správ a príklady 4

REGISTER 4

ACK 4

REJECT 5

DATA 5

RETRANSMIT_REQUEST 6

KEEPALIVE 6

KEEPALIVE_ACK 6

Kontrola integrity a spoľahlivosť 7

Kontrola aktivity senzorov 7

Implementácia servera a testera **8**

Použité knižnice 8

Server – cyklus prijímania 8

Server – cyklus kontroly aktivity 9

Tester – cyklus prijímania 10

Tester – cyklus odosielania 11

Logovanie 12

Akceptačné testovanie UAT **13**

UAT1 – Registrácia a automatické DATA 13

UAT2 – Vlastná dátová správa 14

UAT3 – Poškodenie DATA 14

UAT4 – IDLE a KEEPALIVE 15

UAT5 – Potlačenie ACK 16

Návrh merania efektivity protokolu **17**

Metodika merania 17

Veľkosť správ a sieťová réžia 18

Výpočet úspory 18

Postup vyhodnotenia 19

Zhodnotenie riešenia **19**

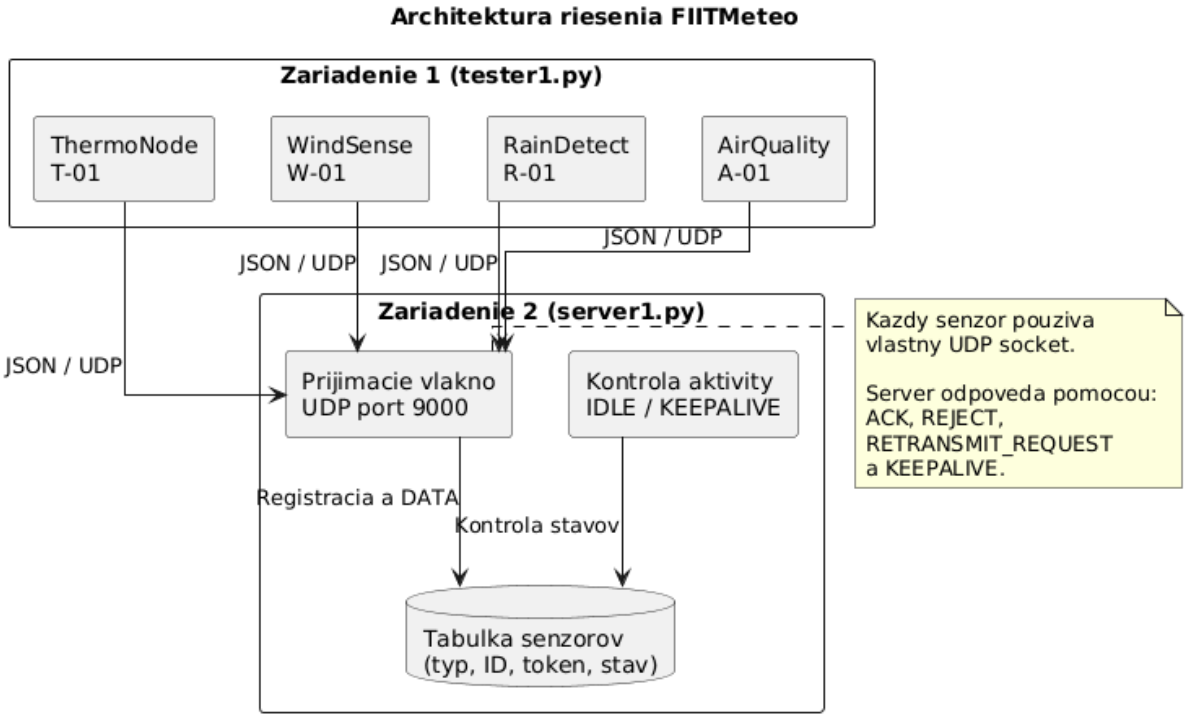
Úvod a architektúra riešenia

Cieľom projektu FIITMeteo je navrhnúť a implementovať aplikačný komunikačný protokol na prenos meteorologických údajov medzi senzormi a centrálnym serverom. Komunikácia prebieha prostredníctvom UDP, pričom v prvej fáze sú správy reprezentované vo formáte JSON.

Riešenie pozostáva z dvoch aplikácií implementovaných v jazyku Python. Server `server.py` prijíma a spracováva správy, registruje senzory, overuje integritu údajov a kontroluje aktivitu zariadení. Tester `tester.py` simuluje štyri typy senzorov: ThermoNode, WindSense, RainDetect a AirQuality.

Každý senzor používa vlastný UDP socket a je identifikovaný kombináciou typu a identifikátora. Server po registrácii prideľuje senzoru token. Predvoleným komunikačným portom je UDP 5055.

Architektúru systému znázorňuje obrázok 1.



Obrázok 1: D1 – Architektúra riešenia FIITMeteo

Návrh komunikačného protokolu

Protokol používa sedem typov správ: REGISTER, ACK, REJECT, DATA, RETRANSMIT_REQUEST, KEEPALIVE a KEEPALIVE_ACK.

Základné polia správ sú `type`, `seq`, `sensor_type`, `sensor_id`, `token`, `ack_for` a `crc`. Konkrétna správa obsahuje iba potrebné polia.

Identifikácia a sekvenčné čísla

REGISTER používa `seq=1`. Každý senzor následne používa vlastné počítadlo sekvenčných čísel začínajúce hodnotou 2. Server používa spoločné počítadlo pre svoje správy.

Pole `ack_for` označuje sekvenčné číslo potvrdzovanej správy. Pri retransmisii sa zachováva pôvodné `seq` aj obsah správy.

Po registrácii server vygeneruje token pomocou `secrets.token_hex(8)`. Token má 16 hexadecimálnych znakov a slúži na overovanie platnosti relácie. Po odpojení sa zneplatní a pri opätovnej registrácii sa vygeneruje nový.

Typy správ a príklady

REGISTER

Registruje senzor podľa jeho typu a identifikátora.

```
{
  "type": "REGISTER",
  "seq": 1,
  "sensor_type": "ThermoNode",
  "sensor_id": "td1"
}
```

ACK

Potvrdzuje registráciu alebo prijatie dát. Pri registrácii obsahuje aj pridelený token.

```
{
  "type": "ACK",
  "seq": 1,
  "ack_for": 1,
  "token": "a1b2c3d4e5f60718"
}
```

Pri potvrdení DATA sa pole `token` neposiela; `ack_for` označuje potvrdzovanú DATA správu.

REJECT

Oznamuje zamietnutie registrácie. Dôvodom môže byť `INVALID_SENSOR` alebo `DUPLICATE_SENSOR`.

```
{
  "type": "REJECT",
  "seq": 3,
  "ack_for": 1,
  "reason": "DUPLICATE_SENSOR"
}
```

DATA

Prenáša namerané hodnoty spolu s tokenom a kontrolným súčtom CRC32.

```
{
  "type": "DATA",
  "seq": 2,
  "sensor_type": "ThermoNode",
  "sensor_id": "td1",
  "token": "a1b2c3d4e5f60718",
  "temperature": 21.3,
  "humidity": 55.4,
  "dew_point": 11.9,
  "pressure": 1013.25,
  "crc": "CRC32"
}
```

Hodnota `crc` je v príklade symbolická. Pri odoslaní obsahuje vypočítaný osemznakový hexadecimálny kontrolný súčet.

Jednotlivé senzory používajú tieto parametre:

- **ThermoNode:** temperature, humidity, dew_point, pressure.
- **WindSense:** wind_speed, wind_gust, wind_direction, turbulence.
- **RainDetect:** rainfall, soil_moisture, flood_risk, rain_duration.
- **AirQuality:** co2, ozone, air_quality_index.

RETRANSMIT_REQUEST

Server požaduje opätovné odoslanie DATA pri nesprávnom CRC32.

```
{
  "type": "RETRANSMIT_REQUEST",
  "seq": 6,
  "ack_for": 5
}
```

KEEPALIVE

Server kontroluje dostupnosť senzora v stave IDLE.

```
{
  "type": "KEEPALIVE",
  "seq": 10,
  "sensor_type": "ThermoNode",
  "sensor_id": "td1",
  "token": "a1b2c3d4e5f60718"
}
```

KEEPALIVE_ACK

Senzor potvrdzuje prijatú KEEPALIVE správu.

```
{
  "type": "KEEPALIVE_ACK",
  "seq": 7,
  "ack_for": 10,
  "sensor_type": "ThermoNode",
  "sensor_id": "td1",
  "token": "a1b2c3d4e5f60718"
}
```

Server overí identitu, token a hodnotu `ack_for`. Platná odpoveď vynuluje počítadlo nezodpovedaných KEEPALIVE správ.

Kontrola integrity a spoľahlivosť

CRC32 sa počíta z JSON správy bez poľa `crc`. Správa sa serializuje pomocou `json.dumps()` s parametrami `sort_keys=True` a `separators=(",", ":")`. Výsledné bajty UTF-8 spracuje `zlib.crc32()`.

Pri nesúlade CRC server odošle RETRANSMIT_REQUEST a tester opakuje pôvodnú nepoškodenú správu.

UDP nezaručuje doručenie, preto tester eviduje nepotvrdené DATA v `pending_messages`. Ak ACK nepríde do 3 sekúnd, správu zopakuje. Maximálny počet pokusov je 3 vrátane prvého. Potom sa správa zahodí a udalosť zaloguje.

Server v aktuálnej implementácii nevykonáva deduplikáciu, preto môže opakovane prijatú platnú DATA správu spracovať viackrát.

Kontrola aktivity senzorov

Server rozlišuje stavy ACTIVE, IDLE a DISCONNECTED. Časovače fungujú nasledovne:

- **ACTIVE:** platná relácia s prijímanými dátami.
- **IDLE:** 15 sekúnd bez platnej DATA správy. Časovač sa neobnovuje prijatím KEEPALIVE_ACK.
- **DISCONNECTED:** tri nezodpovedané KEEPALIVE kontroly; token sa zneplatní.

Server kontroluje senzory približne každú sekundu. V stave IDLE odosiela KEEPALIVE v intervaloch 5 sekúnd. Pri úspešnej odpovedi zostáva senzor IDLE, až kým nepríde nová platná DATA správa.

Pri ideálnom časovaní dôjde k odpojeniu približne po 30 sekundách od poslednej DATA:

$$15 + 5 + 5 + 5 = 30\,\text{s}$$

Tester samostatne kontroluje prijímanie KEEPALIVE. Pri vypnutom automatickom odosielaní a po prijatí aspoň jednej KEEPALIVE správy ukončí lokálnu reláciu, ak ďalšia KEEPALIVE nepríde počas 30 sekúnd.

Implementácia servera a testera

Použité knižnice

Serverová aj testovacia aplikácia sú implementované v jazyku Python a využívajú rovnaké štandardné knižnice:

- `socket` – UDP komunikácia.
- `json` – serializácia správ.
- `zlib` – výpočet CRC32.
- `threading` – súbežné vlákna.
- `time` – časovače a intervaly.
- `random` – generovanie meraní.
- `datetime` – časové značky logov.
- `secrets` – generovanie autentifikačných tokenov (iba server).

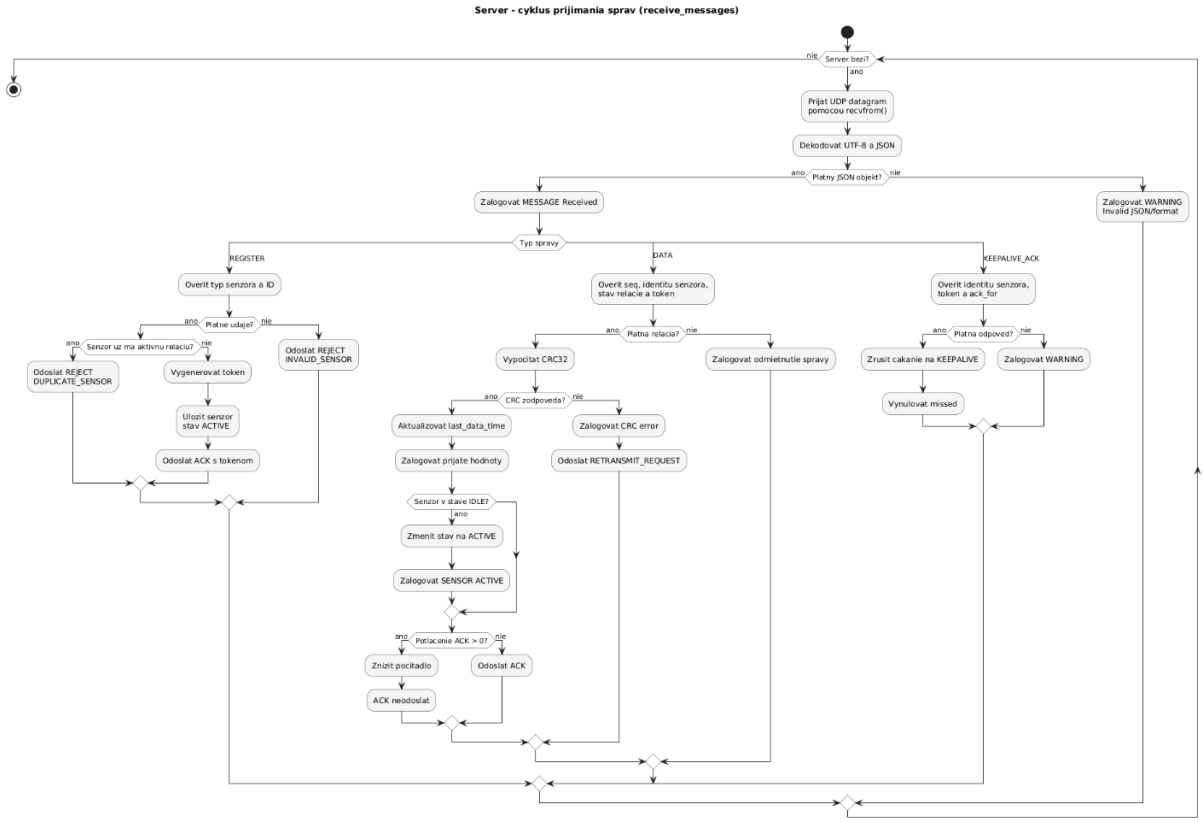
Obrázok 2 zobrazuje knižnice importované v oboch aplikáciách.

```
import socket
import json
import threading
import secrets
import zlib
import time
from datetime import datetime, timezone
```

Obrázok 2: Importované knižnice servera a testera

Server – cyklus prijímania

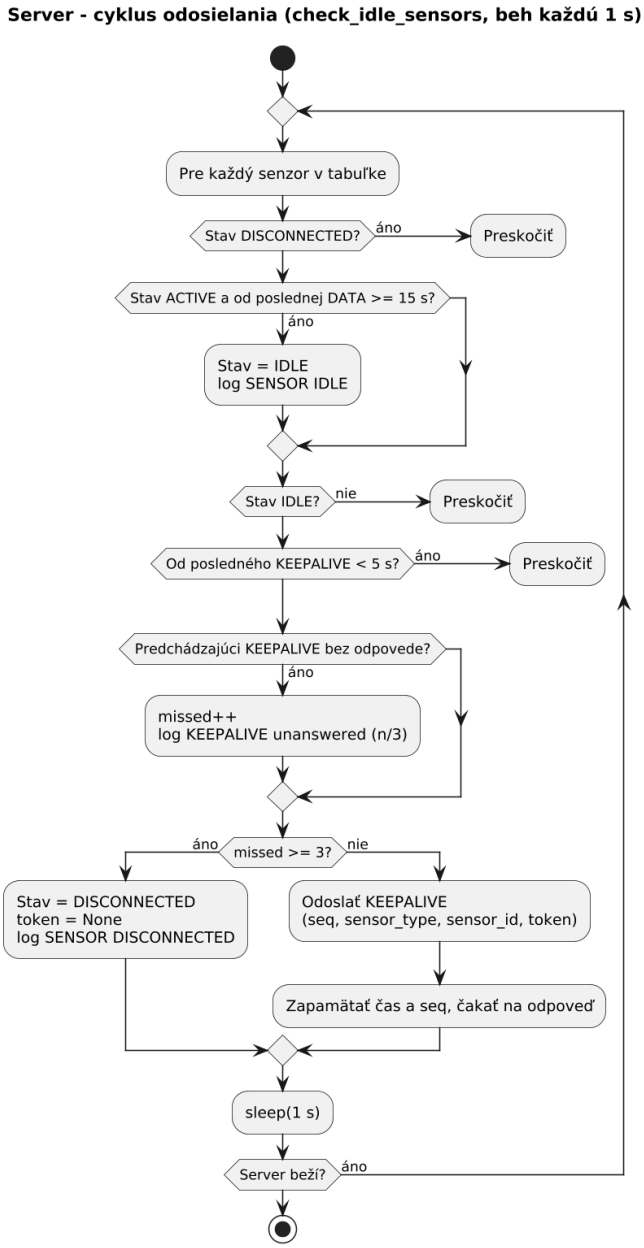
Prijímacie vlákno `server.py` používa `recvfrom()` na prijatie UDP datagramu. Po dekódovaní JSON spracuje REGISTER, DATA alebo KEEPALIVE_ACK. Pri registrácii vytvára reláciu a prideľuje token; pri DATA kontroluje identitu, token a CRC32 a podľa výsledku odosiela ACK alebo RETRANSMIT_REQUEST.



Obrázok 3: D2 – Server: cyklus prijímania správ

Server – cyklus kontroly aktivity

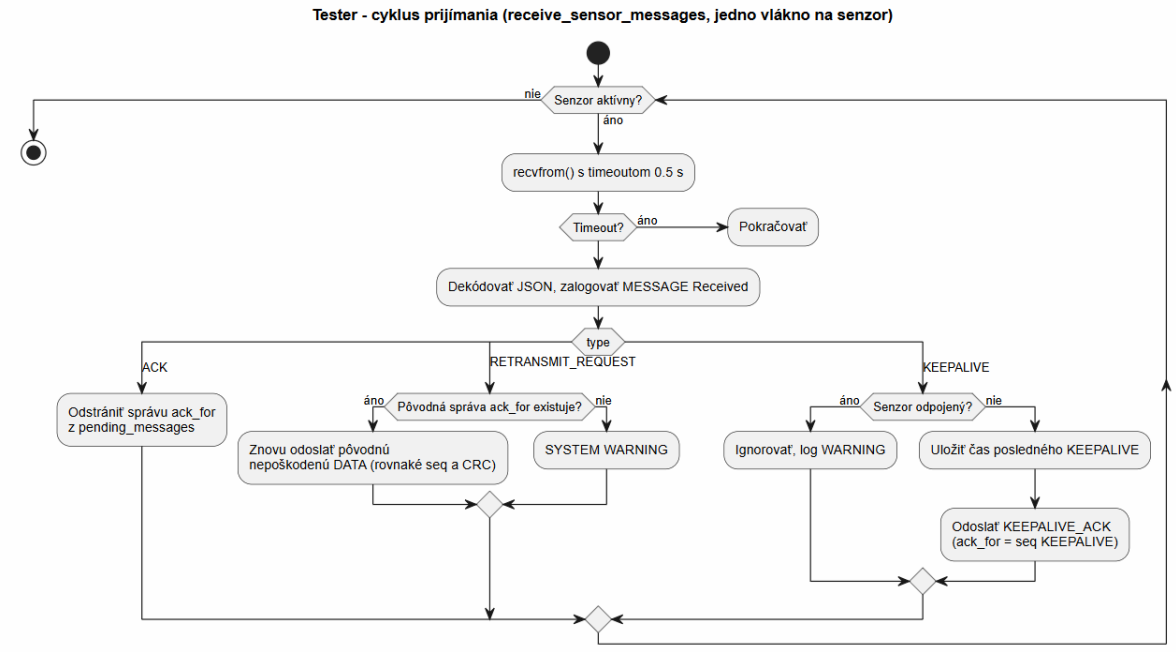
Samostatné vlákno `check_idle_sensors` približne každú sekundu vyhodnocuje čas od poslednej DATA správy. Riadi prechody ACTIVE, IDLE a DISCONNECTED, odosiela KEEPALIVE a vyhodnocuje odpovede.



Obrázok 4: D3 – Server: cyklus odosielania a kontroly aktivity

Tester – cyklus prijímania

Každý simulovaný senzor používa vlastný UDP socket a prijímacie vlákno. Pri ACK odstráni potvrdenú DATA správu z `pending_messages`. Pri RETRANSMIT_REQUEST odošle pôvodné nepoškodené DATA a pri KEEPALIVE odpovie správou KEEPALIVE_ACK. Prijímací socket používa timeout 0,5 sekundy.

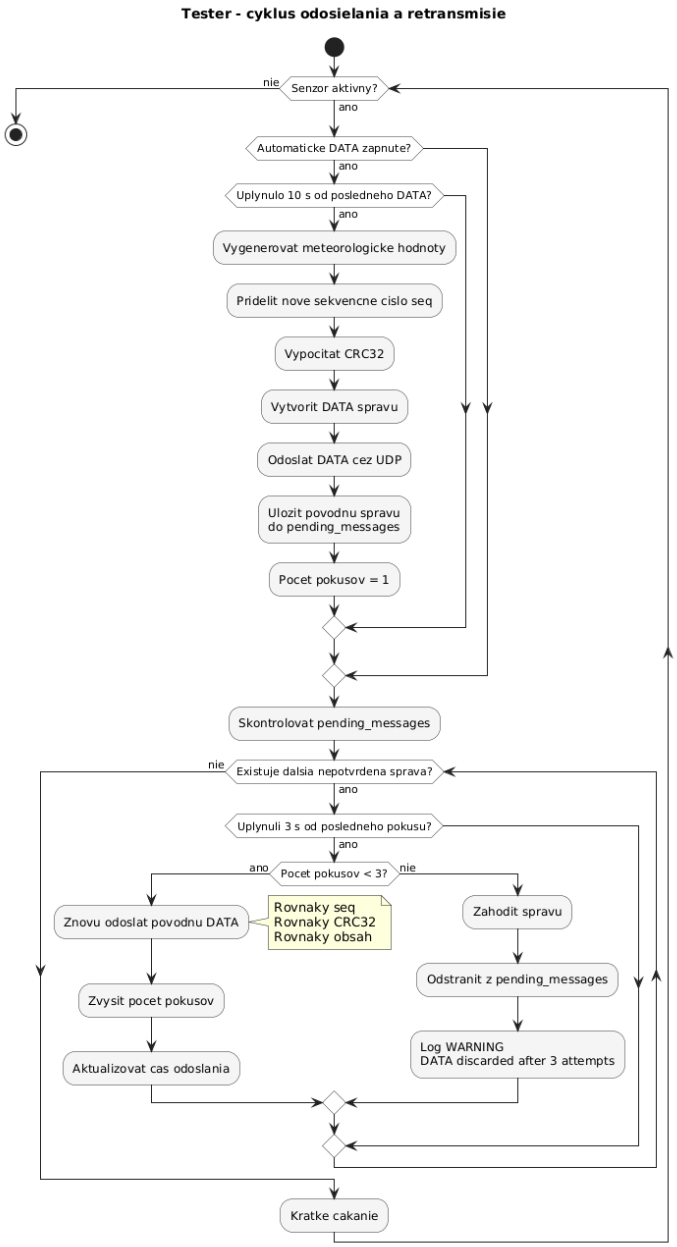


Obrázok 5: D4 – Tester: cyklus prijímania správ

Tester – cyklus odosielania

Tester umožňuje registráciu senzorov, automatické odosielanie, zadanie vlastných hodnôt, poškodenie DATA a simuláciu odpojenia.

Pri automatickom režime vytvára DATA každých 10 sekúnd. Pridelí sekvenčné číslo, vypočíta CRC32, odošle správu a eviduje ju do prijatia ACK. Nepotvrdené správy opakuje podľa nastaveného časovača.



Obrázok 6: D5 – Tester: cyklus odosielania a retransmisie

Logovanie

Server aj tester zapisujú udalosti do súborov `server.log` a `tester.log`. Formát záznamu je:

```
TIMESTAMP | TYPE | LEVEL | SOURCE | MESSAGE
```

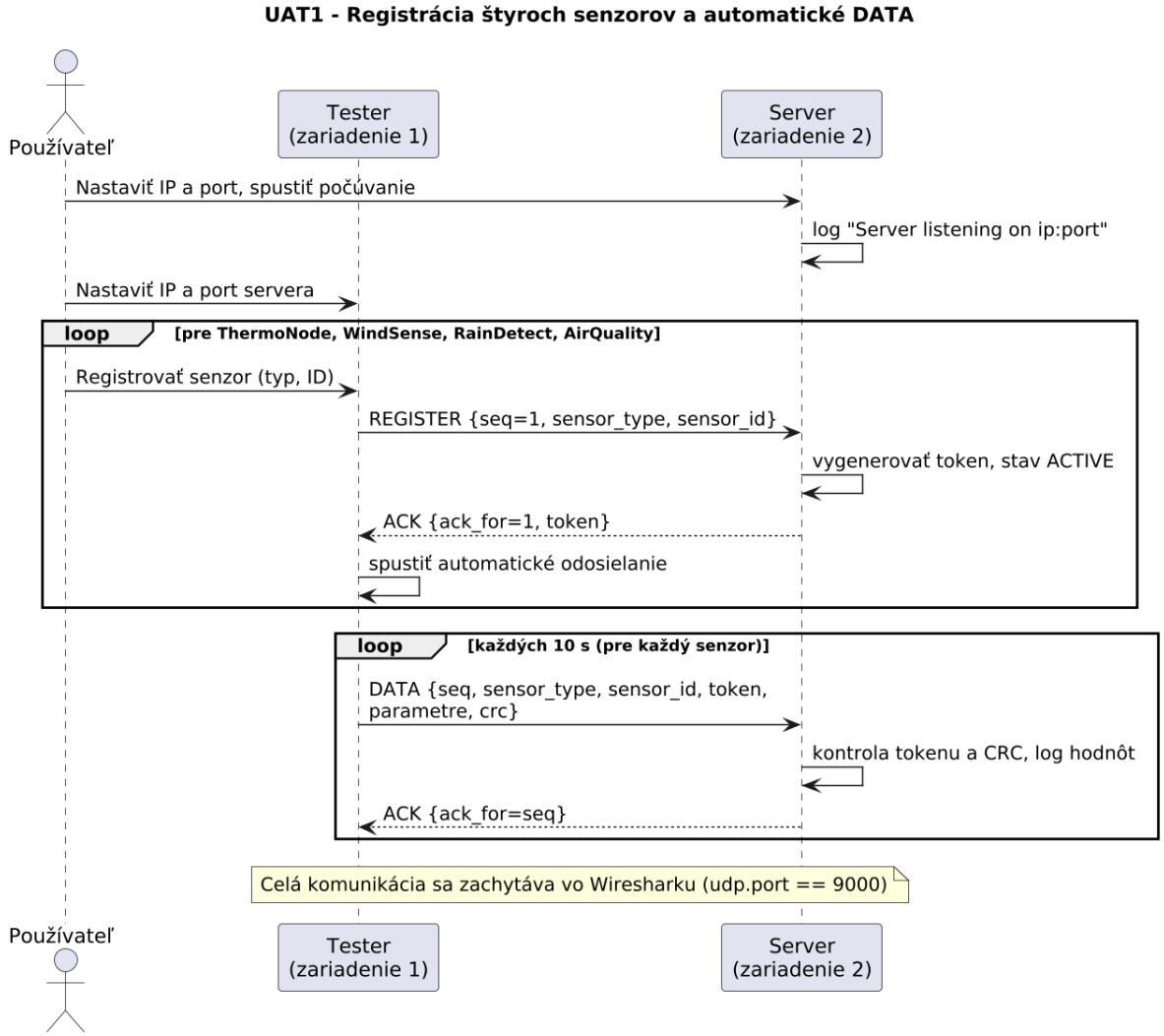
Používajú sa typy SYSTEM a MESSAGE a úrovne INFO, WARNING a ERROR. Časové značky sú v UTC s milisekundami. Výpis do konzoly je v aktuálnej implementácii zapnutý pomocou `console_logging=True`.

Akceptačné testovanie UAT

Akceptačné scenáre overujú komunikáciu medzi serverom a testerom. Testovanie je navrhnuté pre dve zariadenia a zachytávanie UDP komunikácie vo Wiresharku s filtrom `udp.port == 5055`.

UAT1 – Registrácia a automatické DATA

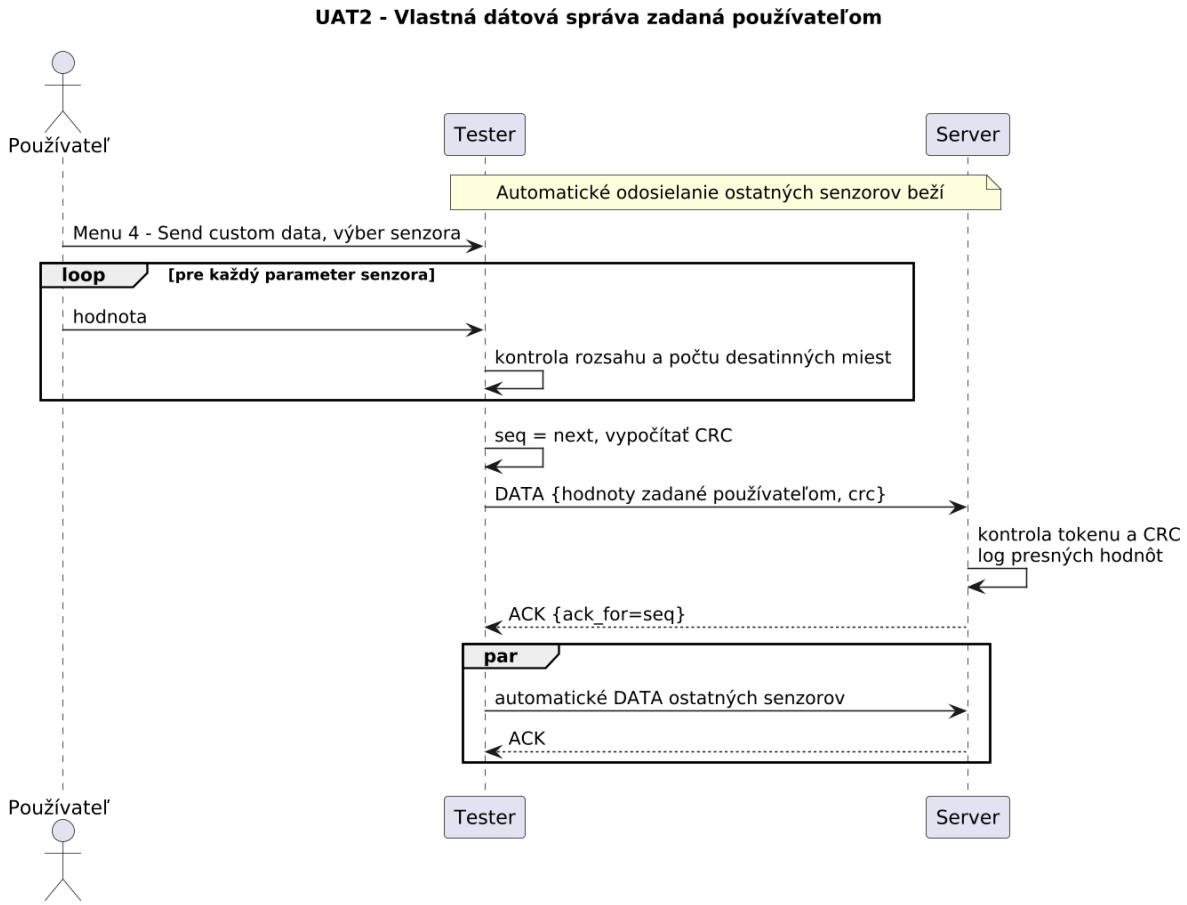
Tester registruje ThermoNode, WindSense, RainDetect a AirQuality. Server každému pridelí token a potvrdí registráciu. Senzory následne odosielajú DATA každých 10 sekúnd a server ich potvrdzuje pomocou ACK.



Obrázok 7: D6 – UAT1: registrácia senzorov a automatické DATA

UAT2 – Vlastná dátová správa

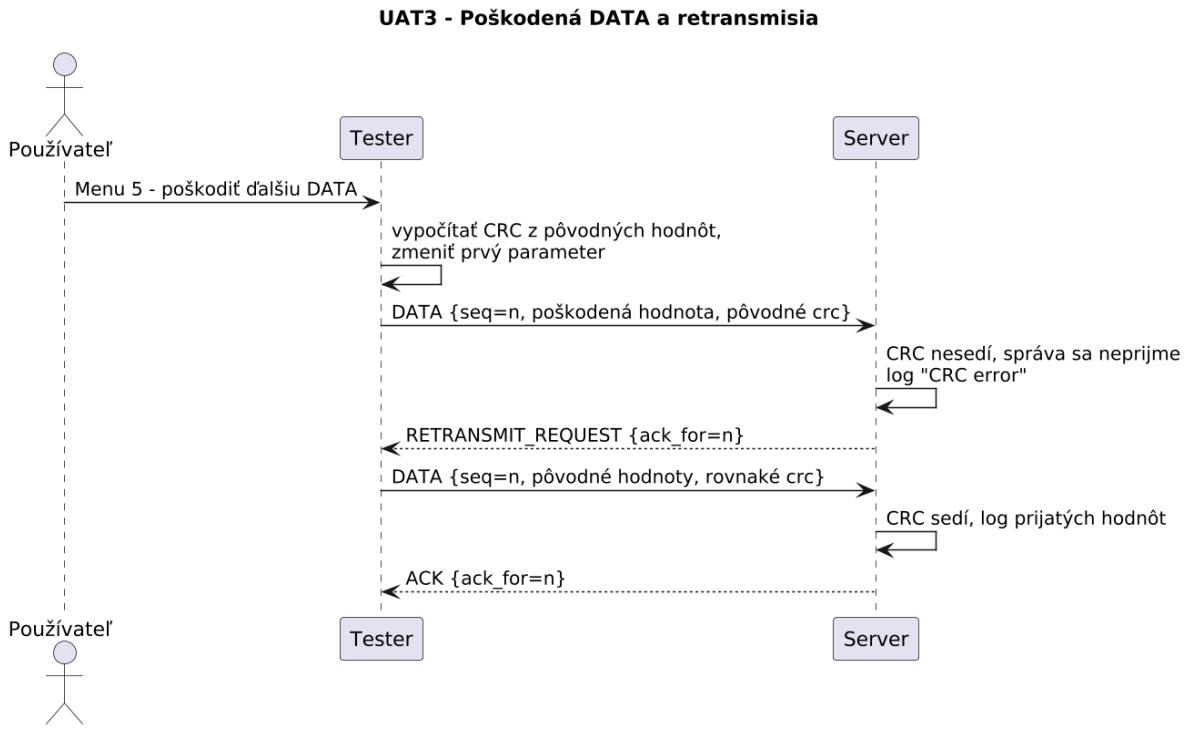
Používateľ vyberie senzor a zadá vlastné meteorologické hodnoty. Tester ich overí, vytvorí DATA s CRC32 a odošle serveru. Server správu spracuje a potvrdí. Ostatné senzory môžu pokračovať v automatickom odosielaní.



Obrázok 8: D7 – UAT2: vlastná dátová správa

UAT3 – Poškodenie DATA

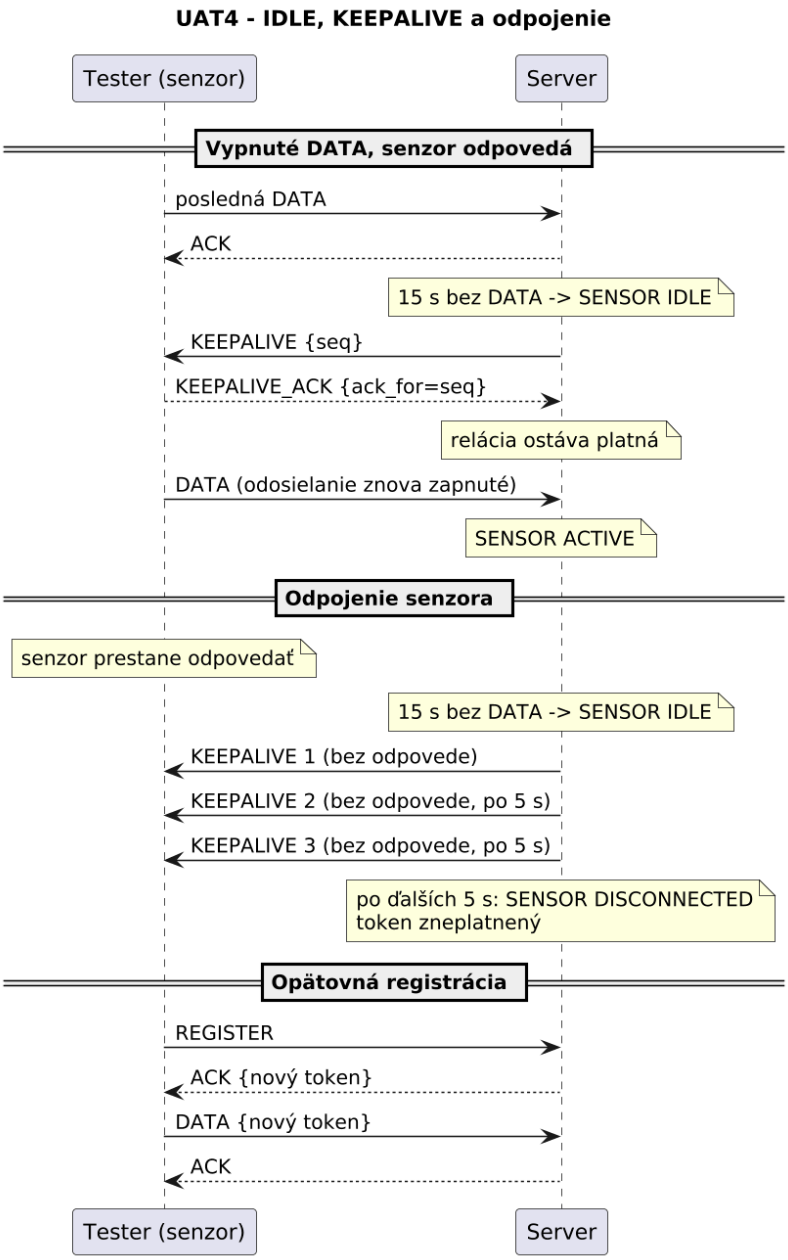
Tester vypočíta CRC32 z pôvodných hodnôt a následne zmení jeden parameter. Server zistí nesúlad a odošle RETRANSMIT_REQUEST. Tester odošle pôvodnú správu s rovnakým sekvenčným číslom a server odpovie ACK.



Obrázok 9: D8 – UAT3: poškodená DATA a retransmisia

UAT4 – IDLE a KEEPALIVE

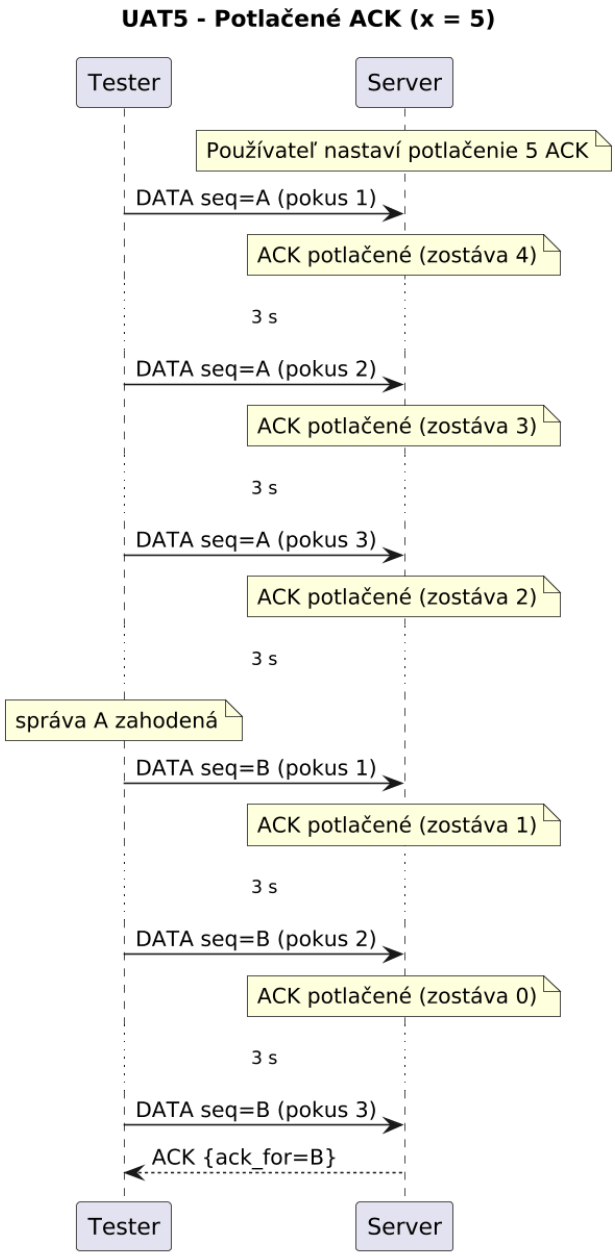
Po 15 sekundách bez DATA prejde senzor do IDLE. Server odosiela KEEPALIVE, na ktoré pripojený senzor odpovedá. Nová DATA obnoví ACTIVE. Pri troch nezodpovedaných kontrolách server zneplatní token a označí reláciu ako DISCONNECTED. Následne je možná opätovná registrácia.



Obrázok 10: D9 – UAT4: IDLE, KEEPALIVE a odpojenie

UAT5 – Potlačenie ACK

Server potlačí päť potvrdení DATA. Prvá správa A je odoslaná trikrát a následne zahodená. Pri ďalšej správe B server potlačí prvé dve potvrdenia, ale tretí pokus už potvrdí pomocou ACK. Test overuje retransmisie po 3 sekundách a maximálne tri pokusy.



Obrázok 11: D10 – UAT5: strata ACK a opakované odosielanie

Návrh merania efektivity protokolu

Metodika merania

Cieľom ďalšej fázy je porovnať objem prenášaných údajov vo formáte JSON a v navrhovanom binárnom formáte.

Oba varianty budú používať rovnaké meteorologické hodnoty, počet správ a testovací scenár.

Komunikácia bude zachytávaná pomocou Wiresharku s filtrom `udp.port == 5055`.

Pre každý typ správy sa porovná veľkosť aplikačných dát aj celkového sieťového rámca. Osobitne sa vyhodnotia DATA správy všetkých štyroch typov senzorov.

Veľkosť správ a sieťová réžia

Veľkosť JSON správy sa určí z počtu skutočne odoslaných UTF-8 bajtov. Pri binárnej verzii sa vypočíta súčet bitových šírok jednotlivých polí vrátane riadiacich a kontrolných údajov.

Pri základnom modeli Ethernet II, IPv4 bez voliteľných polí a UDP uvažujeme nasledujúcu réžiu:

- Ethernet hlavička: 14 B,
- IPv4 hlavička: 20 B,
- UDP hlavička: 8 B.

Súčet hlavičiek je 42 B. Ethernet vyžaduje minimálne 64 B vrátane 4 B FCS. Bez FCS preto platí:

$$S_{\text{ramec}} = \max(60,\ S_{\text{sprava}} + 42)$$

Vzorec zohľadňuje aj prípadný Ethernet padding. FCS, preambula a medzirámcová medzera nie sú v uvedenej veľkosti zahrnuté.

Výpočet úspory

Percentuálnu úsporu vypočítame podľa vzorca:

$$U = \frac{S_{\text{JSON}} - S_{\text{BIN}}}{S_{\text{JSON}}} \cdot 100\,\%$$

Výpočet sa vykoná samostatne pre aplikačné dáta a pre veľkosť rámcov.

Okrem úspory sa vyhodnotí aj efektivita prenosu, teda pomer veľkosti užitočných meteorologických údajov k celkovej veľkosti prenášaného rámca:

$$E = \frac{S_{\text{uzitocne}}}{S_{\text{ramec}}} \cdot 100\,\%$$

Postup vyhodnotenia

Po implementácii binárnej verzie sa počas rovnakého časového intervalu zachytí komunikácia oboch variantov. Vo Wiresharku sa zaznamená počet paketov, objem aplikačných dát a celkový objem prenesených rámcov.

Výsledky sa porovnajú pre jednotlivé typy správ aj pre celý testovací scenár. Následne sa vypočíta percentuálna úspora a vytvoria porovnávacie grafy.

Keďže binárna verzia zatiaľ nie je implementovaná, táto kapitola opisuje plán merania, nie namerané výsledky.

Zhodnotenie riešenia

V prvej fáze projektu bol implementovaný aplikačný protokol FIITMeteo využívajúci UDP a JSON. Podporuje registráciu senzorov, prenos meteorologických údajov, kontrolu CRC32, potvrdzovanie, retransmisie a sledovanie aktivity.

Funkčnosť riešenia je opísaná scenármi UAT1–UAT5. Obmedzením aktuálnej implementácie je chýbajúca deduplikácia prijatých DATA správ na serveri. V ďalšej fáze bude navrhnutá binárna reprezentácia protokolu, Lua dissector pre Wireshark a vykonané meranie efektivity.